

Prova Técnica — Desenvolvimento de Sistemas

Front-end com Banco de Dados · Stack: HTML · CSS · JavaScript · Neon.tech

DURAÇÃO

3 horas

PONTUAÇÃO TOTAL

100 pontos

NOTA PARA APROVAÇÃO

60 pontos

CONSULTA

Somente MDN e Neon Docs

Nome: _____

Turma: _____

Nota: _____



CONTEXTO E MISSÃO

A empresa de serviços **IdeaLab Corp** quer transformar a cultura interna: toda sugestão de melhoria vivia em papéis colados na porta do RH e sumia antes de chegar à diretoria. A caixa de sugestões física foi aposentada e o board decidiu digitalizar o processo.

Você foi designado como o **Desenvolvedor Front-end** responsável por criar a **versão 1.0 do Mural de Sugestões Digital**. O sistema deve ser uma *Single Page Application (SPA)* limpa e responsiva, onde qualquer colaborador possa **enviar** uma sugestão informando seu nome, a mensagem e a área da empresa, **visualizar** todas as sugestões já enviadas, **editar** um registro e **removê-lo** — tudo em uma única tela.

A arquitetura do banco de dados (PostgreSQL) já foi provisionada na nuvem pela equipe de DevOps via **Neon.tech**. Sua missão é construir a interface e conectá-la ao banco via API HTTP.

LAYOUT DE REFERÊNCIA

◆ WIREFRAME ESPERADO — USE COMO GUIA DE ESTRUTURA (NÃO PRECISA SER IDÊNTICO)

Nova Sugestão

Seu Nome *

Mensagem *

Área

Atendimento ▾

 Enviar Sugestão

Sugestões Enviadas

[Atualizar](#)

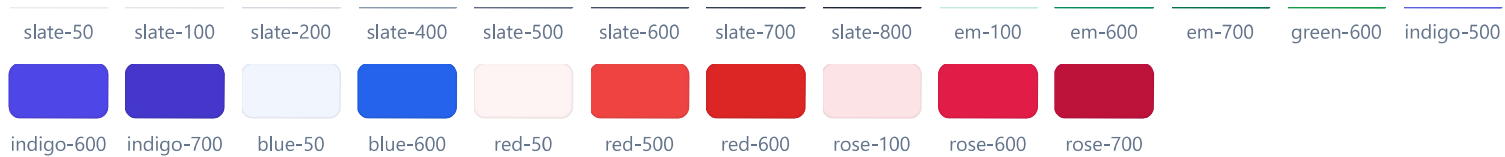
ID	AUTOR / MENSAGEM	ÁREA	AÇÕES
#1		Atend.	 
#2		TI	 
#3		RH	 

1 Layout dividido em dois painéis: formulário (esquerda) + listagem com ações (direita). Cada área possui uma cor de badge distinta — definidas via classes CSS e aplicadas dinamicamente no JS por um objeto de mapeamento. A mensagem é truncada na exibição da tabela para não quebrar o layout.

PALETA DE CORES OFICIAL & MAPEAMENTO DE BADGES

Utilize **obrigatoriamente** as variáveis CSS abaixo. Declare-as em `:root {}` no início do seu CSS.





```
:root {
  --slate-50: #f8fafc;  --slate-100: #f1f5f9;  --slate-200: #e2e8f0;
  --slate-400: #94a3b8;  --slate-500: #64748b;  --slate-600: #475569;  --slate-700: #334155;  --slate-800: #1e293b;
  --emerald-100: #d1fae5;  --emerald-600: #059669;  --emerald-700: #047857;  --green-600: #16a34a;
  --indigo-500: #6366f1;  --indigo-600: #4f46e5;  --indigo-700: #4338ca;
  --blue-50: #eff6ff;  --blue-600: #2563eb;
  --red-50: #fef2f2;  --red-500: #ef4444;  --red-600: #dc2626;
  --rose-100: #ffe4e6;  --rose-600: #e11d48;  --rose-700: #be123c;
}
```

Defina no `style.css` uma classe para cada área. Resultado visual esperado:

Atendimento `.badge-atendimento`
TI `.badge-ti`
RH `.badge-rh`
Infraestrutura `.badge-infra`
Comunicação `.badge-comunicacao`
Outros `.badge-outros`

```
/* Exemplo de implementação - adapte ao seu CSS */
.badge-area { display: inline-flex; align-items: center; padding: 2px 10px; border-radius: 9999px;
font-size: 0.75rem; font-weight: 700; }
.badge-atendimento { background: var(--emerald-100); color: var(--emerald-700); }
.badge-ti { background: var(--blue-50); color: var(--blue-600); }
.badge-rh { background: var(--rose-100); color: var(--rose-700); }
.badge-infra { background: var(--slate-200); color: var(--slate-700); }
.badge-comunicacao { background: var(--blue-50); color: var(--indigo-700); }
.badge-outros { background: var(--slate-100); color: var(--slate-600); }
```



DADOS DO BANCO DE DADOS

O banco já está criado. O professor fornecerá o endpoint e o token antes do início. Preencha abaixo ao receber:

CAMPO	VALOR
Endpoint HTTP (Neon)	https://_____.neon.tech/sql/v1/query
Token (Bearer)	_____
Tabela	sugestoes
Colunas	id (serial PK) · autor (varchar) · mensagem (text) · area (varchar: 'Atendimento', 'TI', 'RH', 'Infraestrutura', 'Comunicação', 'Outros') · enviado_em (timestamp DEFAULT NOW()) — gerado pelo banco)



TAREFAS E REQUISITOS

1

Estrutura Visual — HTML e CSS

Sem frameworks CSS · Layout com Grid ou Flexbox · 4 arquivos

20 pontos

O projeto deve ser organizado em **4 arquivos separados**, cada um com responsabilidade própria:

ARQUIVO	RESPONSABILIDADE
index.html	Estrutura HTML da página, importação dos outros arquivos
style.css	Toda a estilização em CSS puro, usando as variáveis da paleta e as 6 classes <code>.badge-*</code>
script_db.js	Apenas a comunicação com o banco (fetch + queries SQL)

ARQUIVO	RESPONSABILIDADE
<code>script.js</code>	Toda a lógica de interface (manipulação do DOM, eventos, toasts)

O `script.js` deve ser carregado no HTML como **módulo ES6** (`type="module"`) e importar as funções do `script_db.js` via `import { ... } from './script_db.js'`. Utilize a biblioteca **Font Awesome 6** (CDN) para os ícones dos botões.

Layout: dois painéis lado a lado com CSS Grid (`1fr 2fr` no desktop). O formulário deve ter `position: sticky` para acompanhar o scroll da tabela.

Formulário deve conter:

- Título dinâmico (muda entre "Nova Sugestão" e "Editar Sugestão")
- Botão **"Cancelar Edição"** — visível apenas no modo edição (classe `hidden` para ocultar)
- Campo oculto `<input type="hidden" id="sugestao-id">` — armazena o ID durante o UPDATE
- Campos: *Seu Nome* (`type="text"` , obrigatório), *Mensagem* (`<textarea>` com `rows="4"` , obrigatório), *Área* (`select` com opções: Atendimento, TI, RH, Infraestrutura, Comunicação, Outros)
- Botão de submit com ícone `fa-lightbulb` e texto dinâmico

Tabela deve ter cabeçalho: ID · Autor/Mensagem · Área · Ações (centralizado). Botão "↻ **Atualizar**" no header do card da tabela, chamando a função de listagem manualmente.

No `style.css`, declare as **6 classes de badge** (conforme especificado na seção de Paleta): `.badge-atendimento`, `.badge-ti`, `.badge-rh`, `.badge-infra`, `.badge-comunicacao` e `.badge-outros`.

🚫 **Proibido:** frameworks CSS (Bootstrap, Tailwind, etc.) e frameworks JS (React, Vue, etc.). CSS3 e JavaScript puro.

No `script_db.js`, declare a **connection string** do Neon e construa o endpoint HTTP extraindo o host dela:

```
const DATABASE_URL = "postgresql://usuario:senha@host/banco?sslmode=require";
const host = new URL(DATABASE_URL).host;
const neonHttpEndpoint = `https://${host}/sql`;
```

Crie a função **auxiliar central** `executarQueryNeon(querySQL, parametros)` que encapsula todo o `fetch()`.

Todas as outras funções CRUD devem chamá-la — nunca repetir o fetch diretamente. O header de autenticação obrigatório é:

```
'Neon-Connection-String': DATABASE_URL
```

A função deve retornar `dados.rows` em caso de sucesso, ou `null` em caso de erro (use `try/catch`). Erros devem ser exibidos no `console.error`.

Implemente e **exporte** a função `consultarDiretoComFetch()` que:

- Executa `SELECT * FROM sugestoes ORDER BY enviado_em DESC` via `executarQueryNeon()`.
- Retorna o array de sugestões, ou `[]` se ocorrer erro (nunca `null` para a tela).

No `script.js`, implemente `criarTabelaSugestoes()` que:

- Chama `consultarDiretoComFetch()` e limpa o `tbody` antes de renderizar.
- Gera cada `<tr>` dinamicamente com: célula de ID (prefixo `#`), célula nome do autor + trecho da mensagem em linhas separadas, badge de área e coluna de ações.
- A mensagem deve ser **truncada** para no máximo 70 caracteres na exibição da tabela: `s.mensagem.length > 70 ? s.mensagem.substring(0, 70) + '...' : s.mensagem`.

- Badge colorida por área: use um **objeto de mapeamento** em JS para converter o valor da área na classe CSS correspondente:

```
const corPorArea = {
  'Atendimento': 'badge-atendimento',
  'TI': 'badge-ti',
  'RH': 'badge-rh',
  'Infraestrutura': 'badge-infra',
  'Comunicação': 'badge-comunicacao',
  'Outros': 'badge-outros'
};

// uso: `badge-area ${corPorArea[s.area] || 'badge-outros'}`
```

- Se não houver registros, exibir linha com `colspan="4"` e mensagem de estado vazio.
- Ser chamada **automaticamente ao carregar a página**.

💡 Os registros retornados pelo Neon estão em `dados.rows`. O endpoint HTTP aceita apenas o método `POST` com `Content-Type: application/json`.

3

Envio de Sugestões — INSERT e UPDATE

25 pontos

Função unificada de envio · Criação e edição no mesmo formulário

No `script_db.js`, implemente e exporte as funções de escrita:

- `insertSugestao(autor, mensagem, area)` — usa `INSERT INTO ... RETURNING *` e retorna o objeto da sugestão criada (ou `null` em erro). O `enviado_em` é gerado automaticamente pelo banco — **não envie esse campo**.

- `sqlAtualizarSugestao(id, autor, mensagem, area)` — usa `UPDATE ... SET ... WHERE id = $4 RETURNING *`. **A ordem dos parâmetros importa:** `[autor, mensagem, area, id]`.

```
// INSERT
"INSERT INTO sugestoes (autor, mensagem, area) VALUES ($1, $2, $3) RETURNING *"
params: [autor, mensagem, area]

// UPDATE
"UPDATE sugestoes SET autor = $1, mensagem = $2, area = $3 WHERE id = $4 RETURNING *"
params: [autor, mensagem, area, id]
```

No `script.js`, crie a função **unificada** `lidarComEnvioDoFormulario(event)` vinculada ao evento `submit` do formulário:

- Chame `event.preventDefault()` para evitar recarregamento.
- Leia o valor do `<textarea>` com `.value.trim()` para remover espaços desnecessários.
- Leia o campo oculto `sugestao-id`: se preenchido → **UPDATE**; se vazio → **INSERT**.
- Após operação bem-sucedida: chamar `limparFormulario()` e `criarTabelaSugestoes()`.
- Em caso de erro: exibir toast de erro e **não limpar** o formulário (permite tentar novamente).

Implemente também `limparFormulario()` que: chama `form.reset()`, limpa o campo oculto `sugestao-id`, restaura o título e o texto do botão, e esconde o botão "Cancelar Edição".

💡 O `<textarea>` é lido da mesma forma que um `<input>`:
`document.getElementById('mensagem').value`. Use `.trim()` para evitar que espaços em branco passem como conteúdo válido.

Toasts (notificações flutuantes):

- Crie a função `mostrarToast(mensagem, tipo)` — o parâmetro `tipo` aceita três valores: `'success'`, `'error'` e `'info'`.
- Cada tipo deve ter cor e ícone (Font Awesome) diferentes:
 - `success` → fundo `--green-600`, ícone `fa-check-circle`
 - `error` → fundo `--rose-600`, ícone `fa-exclamation-triangle`
 - `info` → fundo `--blue-600`, ícone `fa-info-circle`
- O toast deve ser adicionado ao `#toast-container` (fixado no canto inferior direito da tela).
- Deve sumir automaticamente após **3 segundos** com animação de saída (opacidade + translate).
- No CSS, implemente a animação de entrada via `@keyframes slideIn` e aplique com a classe `toast-enter`.

Função de Exclusão — DELETE:

- No `script_db.js`: exporte `sqlDeletarSugestao(id)` com query `DELETE FROM sugestoes WHERE id = $1 RETURNING *`. Retorna o objeto deletado ou `null`.
- No `script.js`: declare `window.deletarSugestao = async function(id)` — o `window.` é obrigatório pois a função é chamada via `onclick` inline no HTML gerado dinamicamente.
- Exibir `confirm()` antes de excluir. Se o usuário cancelar, não fazer nada.
- Após exclusão bem-sucedida: toast de sucesso + recarregar tabela.

Função de Edição — prepararEdicao():

- Declare `window.prepararEdicao = function(id, autor, mensagem, area)` — também em `window` pelo mesmo motivo.
- Deve preencher o campo oculto `sugestao-id` com o ID recebido (ativa o modo UPDATE).

- Preencher todos os campos: nome do autor no campo de texto, **mensagem completa** no `<textarea>` (não o trecho truncado), área selecionada no `<select>` via `.value = area`.
- Atualizar o título para "Editar Sugestão" e o texto do botão para "Salvar Edição".
- Exibir o botão "Cancelar Edição" (remover classe `hidden`).

Botões de ação nas linhas da tabela:

- Botão editar: fundo `--blue-50`, cor `--blue-600`, ícone `fa-pen`. No hover: fundo `--blue-600`, texto branco.
- Botão excluir: fundo `--red-50`, cor `--red-600`, ícone `fa-trash`. No hover: fundo `--red-600`, texto branco.
- Ambos os botões circulares (`border-radius: 9999px`), tamanho `2rem × 2rem`.
- Os `onclick` devem passar os dados completos da linha. Atenção: a mensagem pode conter aspas — use `data-*` attributes ou escape adequado: `onclick="prepararEdicao(${s.id}, '${s.autor.replace(/'/g, '\\\'')}', this)"` passando a mensagem via atributo `data-mensagem` no botão para evitar quebra de string.

 **Atenção com aspas na mensagem:** como a `mensagem` é um texto livre, ela pode conter aspas simples ou duplas que quebram o `onclick`. A solução mais robusta é armazenar a mensagem em um atributo `data-mensagem` no botão e recuperá-la via `button.getAttribute('data-mensagem')` dentro de `prepararEdicao()`.

 Funções chamadas via `onclick=""` em HTML gerado pelo JS precisam estar no escopo global (`window.nomeDaFuncao`). Funções internas do módulo ES6 não são acessíveis por `onclick` sem isso.



CRITÉRIOS DE AVALIAÇÃO

CRITÉRIO	PESO	NOTA OBTIDA
TAREFA 1 — ESTRUTURA VISUAL (HTML + CSS)		

CRITÉRIO	PESO	NOTA OBTIDA
Projeto separado em 4 arquivos (index.html, style.css, script.js, script_db.js) com type="module" e FontAwesome importados	5 pts	_____
Layout em CSS Grid (1fr 2fr no desktop), formulário com position: sticky, responsivo em mobile	5 pts	_____
Formulário com campo oculto sugestao-id, botão "Cancelar" oculto, título dinâmico; campo mensagem como <textarea> e area como select com 6 opções	5 pts	_____
Variáveis CSS da paleta declaradas; 6 classes .badge-* definidas corretamente no CSS, sem frameworks; hover nas linhas e botões	5 pts	_____
TAREFA 2 — MÓDULO DE BANCO E SELECT		
Função central executarQueryNeon() com Neon-Connection-String no header, try/catch, retorna dados.rows ou null	8 pts	_____
Endpoint construído via new URL(DATABASE_URL).host — sem hardcode do host	4 pts	_____
consultarDiretoComFetch() exportada; criarTabelaSugestoes() renderiza badges coloridas por área via objeto de mapeamento, trunca mensagem com substring(), estado vazio e carrega ao abrir	8 pts	_____
Funções CRUD exportadas no script_db.js e importadas no script.js com sintaxe ES6 (import { ... } from)	5 pts	_____
TAREFA 3 — INSERT E UPDATE (FUNÇÃO UNIFICADA)		
insertSugestao(autor, mensagem, area) com query parametrizada (\$1, \$2, \$3) e RETURNING * — sem enviar enviado_em	8 pts	_____

CRITÉRIO	PESO	NOTA OBTIDA
<code>sqlAtualizarSugestao(id, autor, mensagem, area)</code> com <code>UPDATE ... RETURNING *</code> , parâmetros na ordem correta ([autor, mensagem, area, id])	7 pts	_____
Função <code>lidarComEnvioDoFormulario()</code> com <code>preventDefault()</code> , leitura do <code>textarea</code> com <code>.trim()</code> , bifurcação pelo <code>sugestao-id</code> , limpeza e reload após sucesso	5 pts	_____
Função <code>limparFormulario()</code> com <code>form.reset()</code> , limpeza do campo oculto, restauração do título, texto do botão e ocultação do botão "Cancelar"	5 pts	_____
TAREFA 4 — UX, DELETE E MODO DE EDIÇÃO		
<code>mostrarToast(mensagem, tipo)</code> com 3 tipos (success/error/info), cores corretas, some em 3s com animação CSS (<code>@keyframes slideIn</code>)	8 pts	_____
<code>window.deletarSugestao()</code> com <code>confirm()</code> , DELETE parametrizado com <code>RETURNING *</code> , toast + reload após sucesso	8 pts	_____
<code>window.prepararEdicao()</code> preenchendo autor, mensagem completa no <code>textarea</code> , área no <code>select</code> via <code>.value</code> , campo oculto, título, texto do botão e botão cancelar; estratégia para escapar aspas na mensagem	9 pts	_____
Botões circulares de editar/excluir em cada linha com hover invertido (cor passa para branco), ambos declarados em <code>window</code>	5 pts	_____
TOTAL	100 pts	_____

O QUE ENTREGAR

- ✓ Os 4 arquivos do projeto: `index.html`, `style.css`, `script.js` e `script_db.js` — compactados em **.zip** ou em repositório público no GitHub.
- ✓ A aplicação será testada abrindo o `index.html` diretamente no navegador (sem servidor local). Ela deve comunicar-se com o banco Neon via a `connection string` configurada no `script_db.js`.
- ✓ Print ou gravação de tela demonstrando as 4 operações: **listagem ao abrir**, **INSERT** de nova sugestão, **UPDATE** via botão editar, e **DELETE** com confirmação — preferencialmente com sugestões de áreas diferentes para demonstrar os badges coloridos.
- ✓ Os toasts devem aparecer visíveis na gravação para cada operação realizada.

⚠ **Nota zero nas tarefas 2, 3 e 4** para projetos que utilizem `localStorage`, arrays estáticos como mock permanente, ou que não executem requisições `fetch` reais ao banco Neon.

Dúvidas sobre `fetch` ou configurações do Neon → pergunte ao professor/avaliador.
Lógica de programação e montagem das queries SQL são de inteira responsabilidade do aluno.

Bom código! 🚀